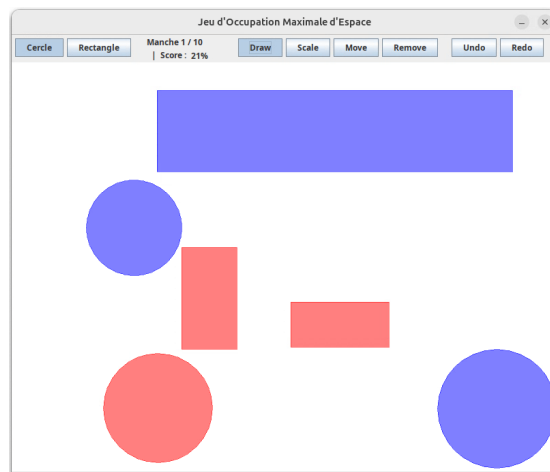


Rapport de projet : Patron et conception

Guillais Marius
David Tom
Pasquier Titouan
Chepy Maxim



Sommaire :

1	Déroulement d'une partie et fonctions disponibles	3
2	Mise en oeuvre des différents patterns	4
2.1	Pattern MVC/Observer	4
2.2	Pattern Strategy	5
2.3	Pattern Abstract Factory	5
2.4	Pattern Composite	5
2.5	Pattern State	5
2.6	Pattern Command	6
2.7	Pattern Double Dispatch	7
2.8	Système de Plug-ins	7
3	Algorithmes non triviaux	8
3.1	Création de forme par un joueur robot	8
3.2	Calcul de score en mode 2 joueurs	8
4	Répartition du travail	9
5	Annexe	10
5.1	Copies d'écrans de l'application	10

1 Déroulement d'une partie et fonctions disponibles

Avant de pouvoir commencer une partie un menu va vous être affiché et vous permettra de choisir différentes options pour la partie telles que :

- Un mode solo et deux joueurs.
- Deux difficultés, avec la plus dure qui fait disparaître les formes au bout de 10 secondes.
- Des générations de formes, soit prédéfinies soit aléatoires

Une fois dans la partie vous avez 10 manches, et à chaque manche vous devez placer 4 formes, une fois ces 4 formes posées vous obtenez un score qui est le pourcentage de l'écran que vous avez réussi à couvrir. Ensuite tout est effacé, puis de nouvelles formes sont posées. Une fois que ce processus a été effectué 10 fois vous obtenez votre score final qui est la moyenne de 10 manches.

Dans le cas du mode 2 joueurs les règles changent. Le premier joueur va placer jusqu'à 4 formes puis ensuite elles disparaissent et le second joueur doit essayer de les recouvrir au maximum. De la même manière que pour le mode solo c'est répété 10 fois avec une moyenne à la fin.

Durant la partie vous avez accès à de nombreuses fonctionnalités.

- Le bouton Draw permet de dessiner avec un drag and drop la forme désirée sur l'écran.
- Les boutons Cercle et Rectangle sur la droite sont pour le choix de la forme à poser.
- Scale, Move et Remove sont respectivement pour redimensionner, déplacer et supprimer une de ses formes.
- Undo et Redo qui permettent respectivement d'annuler la dernière action réalisée et de reproduire la dernière action annulée.
- Dans le mode solo en simple il y a également accès à un bouton IA qui permet de faire comme si une IA générerait les formes selon un algorithme meilleur que l'aléatoire. Cela peut être utile pour voir si l'IA fait un meilleur score que nous.
- Il y a également le score de la manche en cours d'affiché ainsi que le numéro de la manche actuelle.

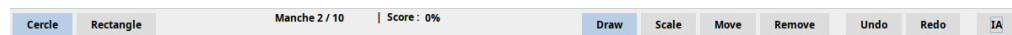
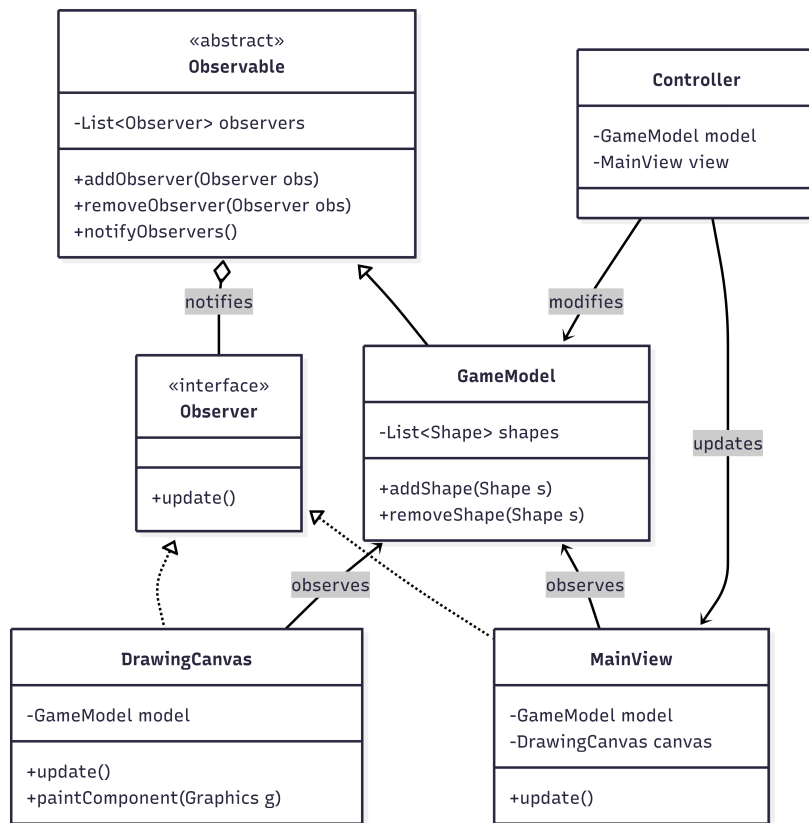


Figure 1: Focus sur la barre avec tout les boutons et infos utiles pour le jeu

2 Mise en oeuvre des différents patterns

2.1 Pattern MVC/Observer

Comme suggéré dans le sujet nous avons utilisé un pattern MVC car ce dernier est adapté à une application contenant une interface graphique. Nous avons simplement séparé notre code en trois packages représentant chacun respectivement le modèle, la vue et le controller. Etant donné que nous avons utilisé le pattern MVC pour l'application le pattern observer a été également implémenté pour pouvoir définir les éléments qui observent et ceux qui sont observés tout cela aidant à avoir un code propre et bien séparé pour le MVC.



2.2 Pattern Strategy

Nous avons utilisé le pattern Strategy pour la génération des niveaux. L'interface **LevelStrategy** définit un unique contrat : **generateLevel()**, qui retourne la liste des formes rouges d'une manche. Deux implémentations concrètes existent : **RandomLevelStrategy**, qui génère aléatoirement des cercles et rectangles sans chevauchement à chaque manche, et **PresetLevelStrategy**, qui rejoue des ensembles de formes pré-enregistrés. Le **GameModel** reçoit la stratégie choisie à sa construction et l'appelle sans jamais connaître son implémentation concrète. Cela permet d'ajouter de nouveaux modes de génération sans toucher au reste du code.

2.3 Pattern Abstract Factory

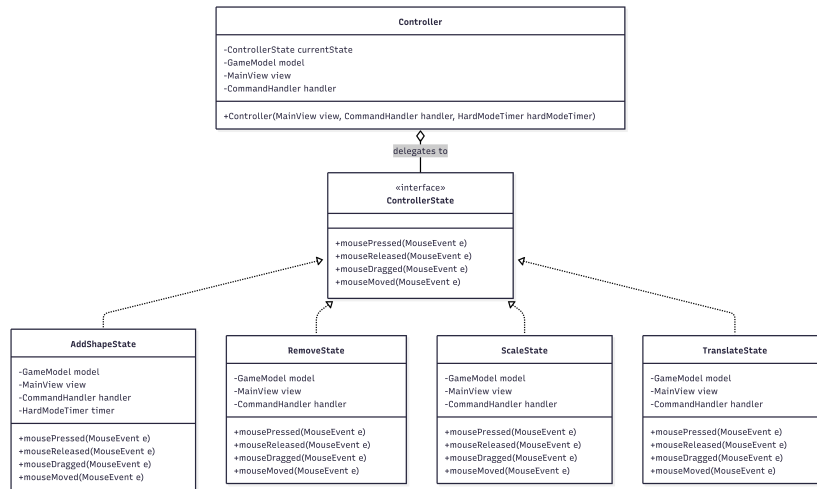
Pour créer les formes bleues, nous avons mis en place une **Abstract Factory**. L'interface **ShapeFactory** expose une méthode **createShape(Point p1, Point p2)** sans préciser le type de forme retourné. Deux fabriques concrètes l'implémentent : **CircleFactory** et **RectangleFactory**. Lorsque l'utilisateur clique sur le bouton "Cercle" ou "Rectangle" dans la barre d'outils, le contrôleur appelle **model.setShapeFactory()** pour remplacer la fabrique active. Le reste du code de création de forme n'est pas modifié : il demande simplement à la factory courante de produire une forme à partir de deux points. Ce découplage permet d'ajouter de nouveaux types de formes en ajoutant uniquement une nouvelle fabrique.

2.4 Pattern Composite

Les formes bleues posées par le joueur sont regroupées dans une classe **GroupeForme** qui implémente elle-même **Shape**. C'est l'application directe du pattern Composite : un groupe se comporte exactement comme une forme individuelle vis-à-vis du reste du code. Chaque opération (**getArea**, **collidesWith**, **translate**, **resize**...) est simplement déléguée à chacun des enfants. Cela permet au **GameModel** de traiter indifféremment une forme isolée ou un groupe, et de calculer le score global en appelant **blueShapes.getArea()** sans itérer manuellement sur la liste.

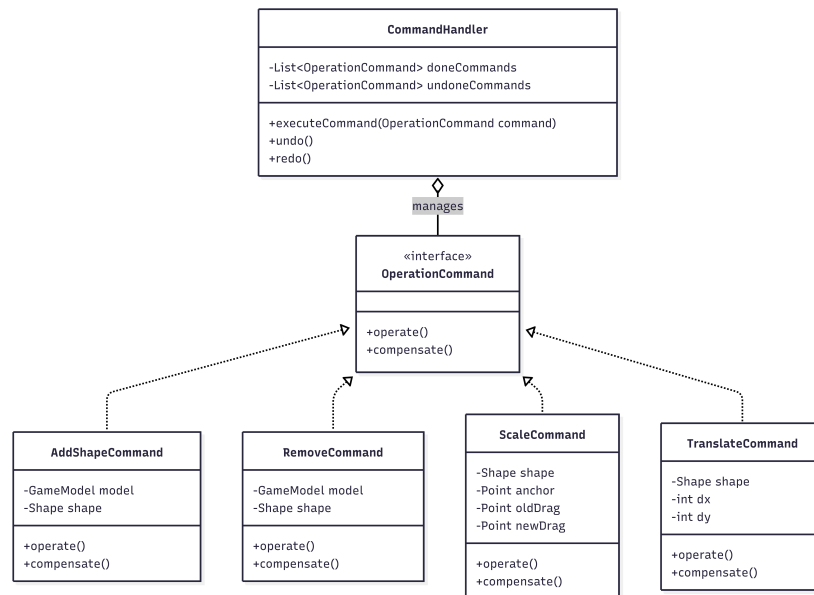
2.5 Pattern State

L'utilisateur va uniquement interagir avec l'application via sa souris. Il y a une multitude d'actions différentes qui peuvent être engendrées, pour pouvoir gérer cela de façon propre nous avons donc utilisé un pattern state. Ce pattern permet au clic de la souris de réaliser une action en fonction de l'état actuel. Nous avons des états pour l'ajout de formes, les redimensionner, les déplacer ou bien pour pouvoir les supprimer.



2.6 Pattern Command

Ce pattern est directement lié au pattern précédent et à une problématique que nous avons eu concernant les "undo/redo". Le fait de pouvoir annuler ou refaire la dernière action. Pour faire les undo/redo nous avons besoin de sauvegarder chaque actions et d'être capable de les annuler tout comme de les reproduire. Grace au pattern command on peut sauvegarder ces 2 informations qui sont complémentaires l'une à l'autre, ainsi quand on va par exemple ajouter une forme on va sauvegarder un command sur cette action, qui va pouvoir selon notre choix annuler cette action ou la refaire dans le cas où on l'aurait supprimé et le pattern command facilite donc énormément ce procéder. Le **CommandHandler** maintient deux piles : les commandes effectuées et les commandes annulées. L'appel à **undo()** dépile la dernière commande effectuée et appelle sa méthode **compensate()**, tandis que **redo()** la replace dans la pile des commandes effectuées. Ce pattern command ainsi que state sont tous deux situés dans le package controller dans leurs packages respectifs : commands et states.



2.7 Pattern Double Dispatch

Le pattern double dispatch nous a été utile pour les collisions entre les formes. En fonction des formes pour lesquelles on calcule si elles entrent en collision nos méthodes ne seront pas les mêmes, ce pattern est donc utile pour cela car facilement outrepasser cela à l'aide de ce pattern. Avant de l'implémenter notre façon pour faire cela était fastidieuse pour quelque chose de pas forcément compliqué à la base. Là c'est très clair car chaque forme va simplement appeler la fonction qui l'intéresse automatiquement quand on regarde si il y a une collision. Concrètement, `Circle.collidesWith(Shape other)` appelle `other.collidesWithCircle(this)`, ce qui résout au moment de l'exécution la combinaison exacte des deux types impliqués.

2.8 Système de Plug-ins

Afin de rendre l'application extensible sans recompilation, nous avons mis en place un système de plug-ins basé sur le chargement dynamique de classes Java. L'interface `PluginDessin` définit le contrat que tout plug-in doit respecter : une méthode `initData(AppContext ctx)` pour recevoir le contexte de l'application, et une méthode `getPanel()` pour fournir un panneau graphique optionnel.

Au démarrage, la classe `PluginLoader` scanne le répertoire `plugins/` à la recherche de fichiers `.jar`. Pour chaque JAR trouvé, elle lit l'attribut `Plugin-Class` dans le `Manifest` du JAR, crée un `URLClassLoader` pointant sur ce fichier, instancie la classe via réflexion et la caste en `PluginDessin`.

L'interface `AppContext` joue un rôle de façade de sécurité : elle expose volontairement un sous-ensemble limité du modèle aux plug-ins (accès aux formes et

au `ThemeManager`), sans leur donner accès au `GameModel` complet. Un plug-in de démonstration, `DarkModePlugin`, illustre ce mécanisme en changeant le thème visuel de l'application via le `ThemeManager`.

3 Algorithmes non triviaux

3.1 Création de forme par un joueur robot

Nous avons un algorithme permettant à un joueur robot de créer un nombre de forme configurable en fonction des besoins. Le but de cet algorithme est de fournir une façon plus intelligente de générer des formes que le hasard pour un joueur robot. Le principe de l'algorithme est que l'on va créer 100 formes aléatoirement puis garder uniquement celle, parmi les 100, qui couvre le plus d'espace. Nous avons plusieurs paramètres cependant pour pallier à de potentiels problèmes comme le fait de générer une trop grosse forme pour pourrait empêcher toutes prochaines formes d'être plaçable. L'opération de choix de formes est répétée jusqu'à avoir le nombre de formes que l'on veut. Un des cas où nous pouvons utiliser cet algorithme peut être à la place de la création de formes via un fichier xml.

3.2 Calcul de score en mode 2 joueurs

Le mode 2 joueurs nécessite un scoring différent du mode solo. Plutôt que de mesurer la surface couverte, il s'agit d'évaluer à quel point le second joueur a reproduit fidèlement les formes du premier. La classe `ScoreCalculator` compare chaque paire de formes (dessinée / reproduite) selon deux critères : la distance euclidienne entre leurs centres, et l'écart de surface entre les deux formes. Chaque critère génère une pénalité plafonnée (80 points max pour la distance, 20 points max pour la surface), ce qui donne un score par paire entre 0 et 100. Le score final de la manche est la moyenne de ces scores par paire, ramené sur le nombre de formes dessinées par le premier joueur.

4 Répartition du travail

Membre	Contributions principales
Tom	Système de plug-ins (<code>PluginLoader</code> , <code>AppContext</code>), Abstract Factory (<code>ShapeFactory</code>), Double Dispatch et <code>ScoreCalculator</code> mode 2 joueurs
Titouan	Pattern State (états souris), architecture MVC, vue (<code>MainView</code> , <code>DrawingCanvas</code> , <code>Toolbar</code>)
Maxim	Pattern Command (<code>CommandHandler</code> , <code>OperationCommand</code>), mécanisme Undo/Redo, <code>TwoPlayerController</code>
Marius	Pattern Observer, Strategy de génération de niveaux (<code>LevelStrategy</code>), gestion des collisions, algorithme IA et rédaction du rapport

Table 1: Répartition du travail entre les membres du groupe

5 Annexe

5.1 Copies d'écrans de l'application

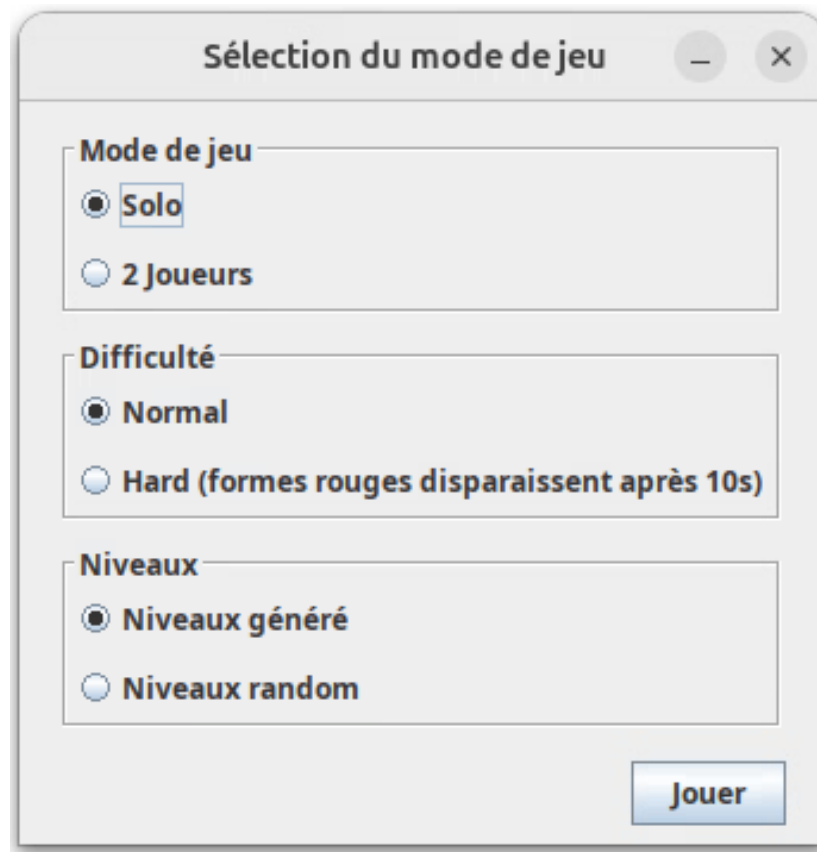


Figure 2: Ecran de démarrage pour sélectionner les options de jeu

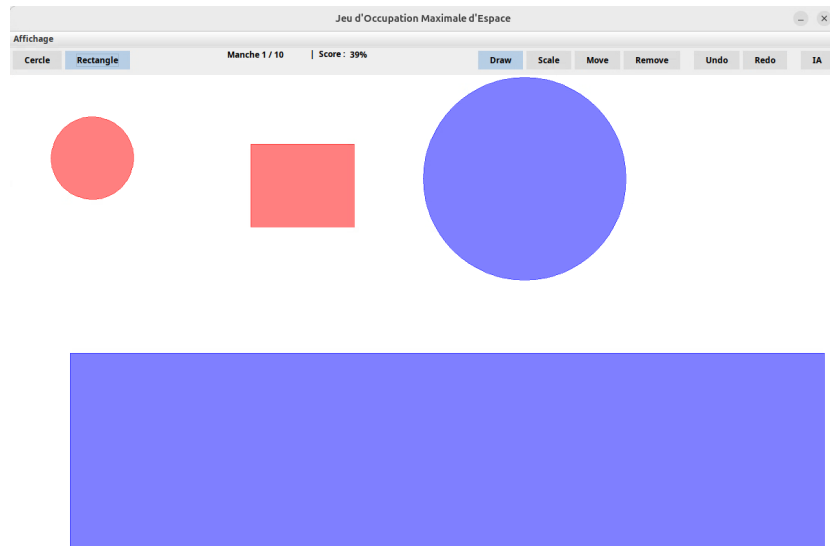


Figure 3: Exemple de partie en cours

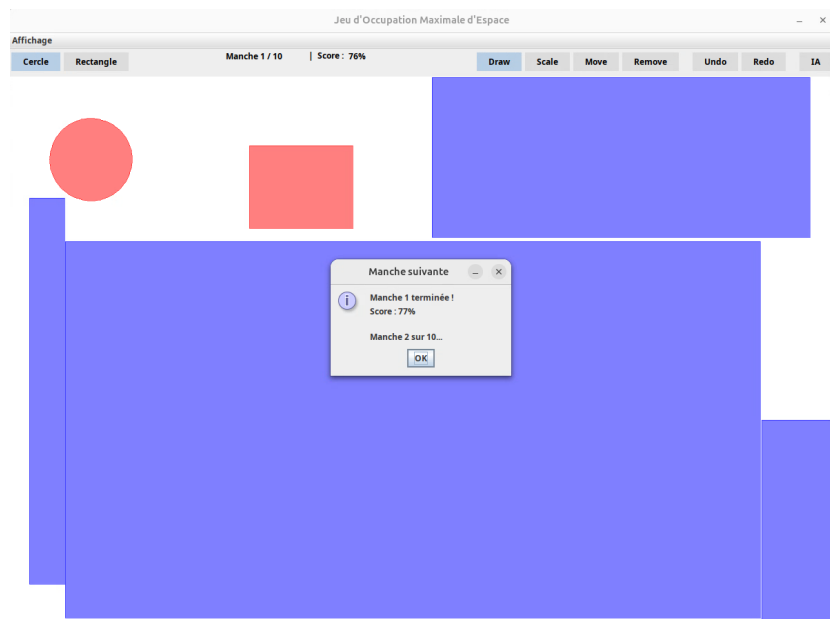


Figure 4: Exemple de résultat d'une manche joué pas IA avec en fond les formes quelle a posé